05
Text Prompt Engineering Techniques
Course · 8 hours ✓ Complete

Course overview

Text Prompt Engineering Techniques

Get Started with Vertex AI Studio

Getting Started with Google Generative AI Using the Gen AI SDK

Common Generative AI Use Cases

Prompt Design Strategies

Generative AI with Vertex AI: Text Prompt Design: Challenge Lab

Your Next Steps

End Lab 00:35:45

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. Learn more.

Open Google Cloud console

GCP Username

student-02-e24a93dc1208

GCP Password

wOs48LHbixu8

GCP Project ID

qwiklabs-gcp-03-2a56866b

Prompt Design Strategies

Lab 1 hour No cost Introductory

Rate Lab

This lab may incorporate AI tools to support your learning.

Lab instructions and tasks

Overview

Objective

Setup and requirements

Task 1. Initialize Vertex AI in a Colab Enterprise notebook

Task 2. Load a generative model

Task 3. Define the output format & specify constraints

Task 4. Assign a persona or role

Task 5. Include examples

Task 6. Experiment with parameter values

Task 7. Utilize fallback responses

Task 8. Add contextual information

Task 9. Structure prompts with prefixes or tags

Task 10. Use system instructions

Task 11. Demonstrate Chain-of-Thought

Task 12. Break down complex tasks

Task 13. Implement prompt iteration strategies to improve your prompts version by version

Congratulations!

Overview

In this lab, you will learn a variety of strategies & techniques to implement when designing prompts for generative models.

You may want to bookmark the following guides to use as references on this topic as you improve your prompt design skills and to stay up-to-date as new techniques are documented:

- An overview of prompting strategies from the Google Cloud documentation.
- A guide to prompt design strategies from the Gemini API documentation.

Objective

In this tutorial, you will see the benefits of implementing some of these strategies.

At a high level, these strategies all involve providing the model **clear and specific instructions** for what output it should produce. In this lab, you will:

- Define the output format & specify constraints
- Assign a persona or role
- Include examples
- Experiment with parameter values
- Utilize fallback responses
- Add contextual information
- Structure prompts with prefixes or tags
- Use system instructions
- Break down complex tasks
- Demonstrate the chain-of-thought
- Implement prompt iteration strategies to improve your prompts version by version

Over your career, you will likely implement & invent new variations on several of these techniques based on your projects' needs.

Setup and requirements

Before you click the Start Lab button

Read these instructions. Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources will be made available to you.

This Qwiklabs hands-on lab lets you do the lab activities yourself in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials that you use to sign in and access Google Cloud for the duration of the lab.

What you need

To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).
- Time to complete the lab.

Note: If you already have your own personal Google Cloud account or project, do not use it for this lab.

Note: If you are using a Pixelbook, open an Incognito window to run this lab.

How to start your lab and sign in to the Google Cloud console

1. Click the **Start Lab** button. If you need to pay for the lab, a dialog opens for you to select your payment method. On the left is the Lab Details pane with the following:

    - The Open Google Cloud console button
    - Time remaining
    - The temporary credentials that you must use for this lab
    - Other information, if needed, to step through this lab

2. Click **Open Google Cloud console** (or right-click and select **Open Link in Incognito Window** if you are running the Chrome browser).

    The lab spins up resources, and then opens another tab that shows the Sign in page.

    Tip: Arrange the tabs in separate windows, side-by-side.

    > **Note:** If you see the **Choose an account** dialog, click **Use Another Account**.

3. If necessary, copy the **Username** below and paste it into the **Sign in** dialog.

```
student-02-e24a93dc1208@qwiklabs.net
```

You can also find the Username in the Lab Details pane.

4. Click **Next**.

5. Copy the **Password** below and paste it into the **Welcome** dialog.

```
w0s48LHbixu8
```

You can also find the Password in the Lab Details pane.

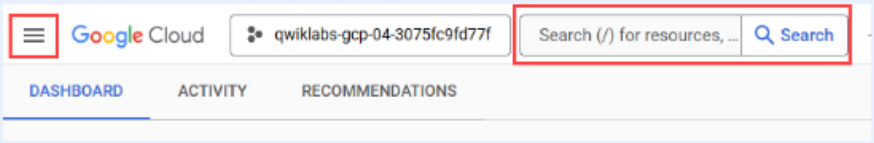
6. Click **Next**.

> **Important:** You must use the credentials the lab provides you. Do not use your Google Cloud account credentials.

> **Note:** Using your own Google Cloud account for this lab may incur extra charges.

7. Click through the subsequent pages:
    - Accept the terms and conditions.
    - Do not add recovery options or two-factor authentication (because this is a temporary account).
    - Do not sign up for free trials.

After a few moments, the Google Cloud console opens in this tab.

> **Note:** To access Google Cloud products and services, click the **Navigation menu** or type the service or product name in the **Search** field.
>
> 
>

Task 1. Initialize Vertex AI in a Colab Enterprise notebook

Create a Colab Enterprise Notebook

1. Navigate to **Colab Enterprise** by searching for it using the search bar located at the top of the console.

2. If prompted, enable required APIs.

3. Click **My notebooks** and select the `us-central1` region from the **Region** drop-down menu, if it is not already selected, and click the **+ New notebook** button to create a new notebook.

4. Click **File > Rename** and rename the notebook to `prompt-design-best-practices.ipynb`.

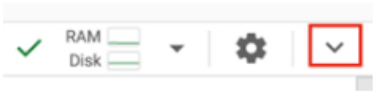
Upgrade the Vertex AI SDK & Restart the Kernel

5. Paste the following code into the top cell and run it with **Shift + Return**. If you don't already have an active notebook runtime, running a cell in a Colab Enterprise notebook will trigger it to create one for you and connect the notebook to it. When a runtime is allocated for the first time, you may be presented with a pop-up window to authorize the environment to act as your Qwiklabs student account.

```
%pip install --upgrade --quiet google-cloud-aiplatform
```

For the **Open OAuth popup** pop-up, click **Open** and select your student username.

6. After the cell completes running, indicated by a checkmark to the left of the cell, the packages should be installed. To use them, we'll restart the runtime. Click on the downward-pointing caret (an icon like an arrow) in the upper right of the notebook:



7. Clicking the caret should have revealed a set of menus above the notebook (File, Edit, View, etc.). Select **Runtime** > **Restart Session** > **select Yes**. The runtime will restart, indicated by clearing the green checkmark and the cell run order integer next to the cell you ran above.

8. Click **+ Code** to create a new code cell and paste in the import & configuration code below. Press **Shift + Return** to run the cell.

```
from inspect import cleandoc
from IPython.display import display, Markdown

import vertexai
from vertexai.generative_models import GenerativeModel,
GenerationConfig
```

Import packages

9. Paste the following into a new code block & run it to initialize Vertex AI.

```
PROJECT_ID = "qwiklabs-gcp-03-2a56866b6f5d"
LOCATION = "us-central1"
import vertexai
vertexai.init(project=PROJECT_ID, location=LOCATION)
```

Task 2. Load a generative model

1. Create a new code cell and run the following to load Gemini Pro.

```
model = GenerativeModel("gemini-2.0-flash-001")
```

Task 3. Define the output format & specify constraints

1. In a new cell, paste & run the following to store a transcript from a customer's order at a fast food restaurant.

```
transcript = """
    Speaker 1 (Customer): Hi, can I get a cheeseburger and large
fries, please?
    Speaker 2 (Restaurant employee): Coming right up! Anything
else you'd like to add to your order?
    Speaker 1: Hmmm, maybe a small orange juice. And could I get
the fries with ketchup on the side?
    Speaker 2: No problem, one cheeseburger, one large fries
with ketchup on the side, and a small
    orange juice. That'll be $5.87. Drive through to the next
window please.
"""
```

2. Run the following prompt that attempts to understand a customer's order from a conversation in JSON format, but in an unspecific way.

```
response = model.generate_content(f"""
    Extract the transcript to JSON.

    {transcript}
""")

print(response.text)
```

```
```json
{
  "turns": [
    {
      "speaker": "Customer",
      "text": "Hi, can I get a cheeseburger and large fries,
please?"
    },
    {
      "speaker": "Restaurant employee",
      "text": "Coming right up! Anything else you'd like to add to
your order?"
    },
    {
      "speaker": "Customer",
      "text": "Hmmm, maybe a small orange juice. And could I get the
fries with ketchup on the side?"
    },
    {
      "speaker": "Restaurant employee",
      "text": "No problem, one cheeseburger, one large fries with
ketchup on the side, and a small orange juice. That'll be $5.87. Drive
through to the next window please."
    }
  ]
}
```
```

3. Now run a version of this prompt with more specific instructions about exactly how you would like your output structured. Notice how the JSON output now reflects the key pieces of information you are interested in to understand the user's order:

```
response = model.generate_content(f"""
    <INSTRUCTIONS>
    - Extract the ordered items into JSON.
    - Separate drinks from food.
    - Include a quantity for each item and a size if specified.
    </INSTRUCTIONS>

    <TRANSCRIPT>
    {transcript}
    </TRANSCRIPT>
""")

print(response.text)
```

```
```json
{
  "food": {
    "cheeseburger": {
      "quantity": 1
    },
    "fries": {
      "quantity": 1,
```
```

```
          "size": "large",
          "notes": "with ketchup on the side"
        }
      },
      "drinks": {
        "orange juice": {
          "quantity": 1,
          "size": "small"
        }
      }
    }
    ```
```

For a real project, you'd go even further in specifying the output by defining the keys that should be included if applicable and likely providing examples.

## Task 4. Assign a persona or role

1. Try an example within the context of a chat. To start, create a chat session with your Gemini model:

```
chat = model.start_chat()
```

2. Ask for a response without a persona specified:

```
response = chat.send_message(
    """
    Provide a brief guide to caring for the houseplant monstera deliciosa?
    """
)

print(response.text)
```

```
## Monstera Deliciosa Care Guide:

**Light:** Bright, indirect light. Avoid direct sunlight.

**Watering:** Allow top 2-3 inches of soil to dry between waterings. Water deeply.

**Humidity:** Moderate to high (around 60%). Mist regularly or group with other plants.

**Temperature:** 65-80°F (18-27°C). Avoid cold drafts.

**Soil:** Well-draining potting mix with high organic matter. Add perlite or coco coir for drainage.

**Fertilizer:** Monthly during spring/summer with balanced liquid
```
```

fertilizer. Reduce to every other month in fall/winter.

Pruning: Prune to control size or shape using sharp shears above a node. Encourages new leaves.

Support: Provide support (moss pole, trellis, stake) as needed to prevent stem drooping.

Pests & Diseases: Check for pests (spider mites, mealybugs, scale) and treat accordingly. Avoid overwatering to prevent root rot.

Additional: Propagate from stem cuttings. Wipe leaves with damp cloth. Repeat every 2-3 years in a larger pot.

Note: This information is based on my knowledge as of November 2023. Please consult reliable sources for the most up-to-date care instructions.

2. Now run a version of this prompt with a role specified. Notice how this output might be more personal and appealing for some users.

```
new_chat = model.start_chat()

response = new_chat.send_message(
    """
    You are a houseplant monstera deliciosa. Help the person who
    is taking care of you to understand your needs.
    """
)

print(response.text)
```

Hello! I'm the Monstera Deliciosa, also known as the Swiss Cheese Plant. I'm a tropical plant that thrives on humidity and indirect sunlight. I'm a relatively low-maintenance plant, but I do have a few needs that you should be aware of.

First, I need to be watered regularly. However, I don't like to be overwatered, so it's important to let the soil dry out slightly between waterings. I also need to be fertilized every few weeks during the spring and summer months. I'm not picky about fertilizer, so you can use any balanced fertilizer that you have on hand.

I like to be kept in a warm, humid environment. If the air is too dry, my leaves can start to brown and curl. You can increase the humidity around me by placing me on a tray of pebbles filled with water. I also appreciate being misted with water occasionally.

As a climbing plant, I need something to climb on. You can provide me with a moss pole, trellis, or even a simple stake. I will eventually grow large and heavy, so it's important to provide me with some support.

If you take care of me properly, I will reward you with beautiful, glossy leaves. I can also produce fruit, but it's not edible for humans.

Here are some additional tips for caring for me:

```
Here are some additional tips for caring for me:

   * Don't report me too often. I prefer to be slightly rootbound.
   * Don't put me in direct sunlight. This can scorch my leaves.
   * If my leaves start to turn yellow, it could be a sign that I'm not
getting enough water or fertilizer.
   * If my leaves start to turn brown, it could be a sign that I'm being
overwatered or that the air is too dry.

I hope this information helps you to understand my needs. With a little
care, I will be a beautiful and rewarding addition to your home.
```

Task 5. Include examples

A prompt that includes no examples is called a **zero-shot** prompt. One with a single example is a **one-shot** prompt. And a few examples would make it a **few-shot** prompt.

1. In the following example, you'd like a model to rate the submissions from a "Contact Us" form on your software development company's website based on how "hot" the lead is (meaning how likely they are to be a valuable customer). In order to help the model understand your rating system, you will provide it examples of different ratings before providing it the customer message you would like evaluated.

2. Run the code below in a new cell to see how it interprets the Customer Message and explains its response.

```
question = """
We offer software consulting services. Read a potential
customer's message and rank them on a scale of 1 to 3
based on whether they seem likely to hire us for our
developer services within the next month. Return the likelihood
rating labeled as "Likelihood: SCORE".
Do not include any Markdown styling.

1 means they are not likely to hire.
2 means they might hire, but they are not likely ready to do
so right away.
3 means they are looking to start a project soon.

Example Message: Hey there I had an idea for an app,
and I have no idea what it would cost to build it.
Can you give me a rough ballpark?
Likelihood: 1

Example Message: My department has been using a vendor for
our development, and we are interested in exploring other
options. Do you have time for a discussion around your
services?
Likelihood: 2

Example Message: I have mockups drawn for an app and a budget
```

```
allocated. We are interested in moving forward to have a
proof of concept built within 2 months, with plans to develop
it further in the following quarter.
Likelihood: 3

Customer Message: Our department needs a custom gen AI solution.
We have a budget to explore our idea. Do you have capacity
to get started on something soon?
Likelihood: """

response = model.generate_content(question)

print(response.text)
```

```
## Likelihood: 3

**Explanation:**

This customer's message exhibits several strong indicators that they are
likely to hire your software consulting services within the next month:

* **Clear need:** They explicitly state their need for a "custom gen AI
solution," indicating a well-defined project in mind.
* **Budget allocated:** Mentioning a budget demonstrates their
preparedness to invest in development.
* **Urgency:**  The phrase "get started on something soon" suggests a
desire to move forward quickly.
* **Openness to collaboration:**  Asking about your capacity implies
they are ready to engage with a consulting firm.

All these factors combined point to a high likelihood of this customer
moving forward with your services in the near future.
```

Task 6. Experiment with parameter values

1. Run the code from the following cell a few times. The **temperature** and **top_p** parameters which lead to variety in responses are set to low values, so the output should be the same, or very close to the same, each time.

```
response = model.generate_content(
    """
    Tell me a joke about frogs.
    """,
    generation_config={"top_p": .05,
                       "temperature": 0.05}
)

print(response.text)
```

```
Why did the frog get sent to the principal's office?
```

```
Because he was caught skipping class!
```

2. Now run this version of the code a few times. You'll see that the higher **temperature** and **top_p** parameter values now lead to more varied results. Some of the results, however, may be a little too random and not make very much sense. If you want variety in your responses, you'll need to experiment with parameters to determine the right balance of creativity and reliability.

```
response = model.generate_content(
    """
    Tell me a joke about frogs.
    """,
    generation_config={"top_p": .98,
                       "temperature": 1}
)

print(response.text)
```

```
    Why don't frogs play poker in the jungle?

    Because they always cheat with their rigged dice!

    Do you want to hear another joke?
```

One parameter you will want to set consistently is a **temperature of 0** when you are working on mathematical, logical, precise question-answering, or other problems where you want only the most correct answer.

Task 7. Utilize fallback responses

When you are building a generative AI application, you may want to restrict the scope of what kinds of queries your application will respond to.

A **fallback response** is a response the model should use when a user input would take the conversation out of your intended scope. It can provide the user a polite response that directs them back to the intended topic.

1. Copy the following code cells to your notebook and run them to see the model decline to talk about off-topic queries while still answering on-topic queries.

```
response = model.generate_content(
    """
    Instructions: Answer questions about pottery.
    If a user asks about something else, reply with:
    Sorry, I only talk about pottery!

    User Query: How high can a horse jump?
    """
```

```
)

print(response.text)
```

```
    Sorry, I only talk about pottery!
```

2. You'll also want to test that the model does not reject on-topic questions, so make sure to try out a variety of questions that the model should not decline to answer. Here is one:

```
response = model.generate_content(
    """
    Instructions: Answer questions about pottery.
    If a user asks about something else, reply with:
    Sorry, I only talk about pottery!

    User Query: What is the difference between ceramic
    and porcelain? Please keep your response brief.
    """
)

print(response.text)
```

```
The main difference between ceramic and porcelain is the material they
are made from. Ceramic is made from clay, while porcelain is made from a
mixture of clay and other minerals, such as feldspar and quartz. This
makes porcelain more durable and translucent than ceramic. Porcelain is
also fired at a higher temperature, which gives it a smoother and more
refined appearance.
```

Task 8. Add contextual information

Imagine you work for a grocery store chain and want to provide users a way of finding items in your store easily.

1. In a new cell, run the following prompt to inquire about the aisle numbers of some items that can be found in a grocery store.

```
response = model.generate_content(
    """
    On what aisle numbers can I find the following items?
    - paper plates
    - mustard
    - potatoes
    """
)

print(response.text)
```

```
## Aisle Numbers:

Here's where you can find the items you listed:

* **Paper plates:** Aisle 7 (usually near plastic cups and cutlery)
* **Mustard:** Aisle 4 (often with other condiments like ketchup and
mayonnaise)
* **Potatoes:** Aisle 2 (look for the vegetable section, near onions and
carrots)

**Please note:** This is a general guide based on typical supermarket
layouts. The actual aisle numbers might vary depending on the specific
store you're visiting. It's always a good idea to check the store
directory or ask a staff member for assistance if you can't find
something.
```

2. The model provides aisle numbers, but these are hallucinations (or "made up" or "imagined" responses) because the model doesn't have the information it needs to provide an accurate response specific to your specific grocery store. Now run a version of this prompt where you provide it that information in what is called the "context" part of the prompt:

```
response = model.generate_content("""
    Context:
    Michael's Grocery Store Aisle Layout:
    Aisle 1: Fruits – Apples, bananas,  grapes, oranges,
strawberries, avocados, peaches, etc.
    Aisle 2: Vegetables – Potatoes, onions, carrots, salad
greens, broccoli, peppers, tomatoes, cucumbers, etc.
    Aisle 3: Canned Goods – Soup, tuna, fruit, beans,
vegetables, pasta sauce, etc.
    Aisle 4: Dairy – Butter, cheese, eggs, milk, yogurt, etc.
    Aisle 5: Meat– Chicken, beef, pork, sausage, bacon etc.
    Aisle 6: Fish & Seafood– Shrimp, crab, cod, tuna, salmon,
etc.
    Aisle 7: Deli– Cheese, salami, ham, turkey, etc.
    Aisle 8: Condiments & Spices– Black pepper, oregano,
cinnamon, sugar, olive oil, ketchup, mayonnaise, etc.
    Aisle 9: Snacks– Chips, pretzels, popcorn, crackers, nuts,
etc.
    Aisle 10: Bread & Bakery– Bread, tortillas, pies, muffins,
bagels, cookies, etc.
    Aisle 11: Beverages– Coffee, teabags, milk, juice, soda,
beer, wine, etc.
    Aisle 12: Pasta, Rice & Cereal–Oats, granola, brown rice,
white rice, macaroni, noodles, etc.
    Aisle 13: Baking– Flour, powdered sugar, baking powder,
cocoa etc.
    Aisle 14: Frozen Foods – Pizza, fish, potatoes, ready meals,
ice cream, etc.
    Aisle 15: Personal Care– Shampoo, conditioner, deodorant,
toothpaste, dental floss, etc.
    Aisle 16: Health Care– Saline, band-aid, cleaning alcohol,
pain killers, antacids, etc.
```

```
    Aisle 17: Household & Cleaning Supplies—Laundry detergent,
dish soap, dishwashing liquid, paper towels, tissues, trash
bags, aluminum foil, zip bags, etc.
    Aisle 18: Baby Items– Baby food, diapers, wet wipes, lotion,
etc.
    Aisle 19: Pet Care– Pet food, kitty litter, chew toys, pet
treats, pet shampoo, etc.

    Query:
    On what aisle numbers can I find the following items?
    - paper plates
    - mustard
    - potatoes
    """
)

print(response.text)
```

```
## Michael's Grocery Store Aisle Numbers

Based on the aisle layout you provided, here's where you can find the
items you listed:

* **Paper plates:** Aisle 17 (Household & Cleaning Supplies)
* **Mustard:** Aisle 8 (Condiments & Spices)
* **Potatoes:** Aisle 2 (Vegetables)
```

3. Notice that the output aisle numbers now correspond to the context you provided.

Task 9. Structure prompts with prefixes or tags

1. Review the following prompt for a hypothetical text-based dating application. It contains several prompt components, including:
    - defining a persona
    - specifying instructions
    - providing multiple pieces of contextual information for the main user and potential matches.
2. Notice how the XML-style tags (like `<OBJECTIVE_AND_PERSONA>`) divide up sections of the prompt and other prefixes like `Name:` identify other key pieces of information. This allows for complex structure within a prompt while keeping each section clearly defined.
3. Copy this code block to a new cell and run it to see the output.

```
prompt = """
  <OBJECTIVE_AND_PERSONA>
  You are a dating matchmaker.
  Your task is to identify common topics or interests between
  the USER_ATTRIBUTES and POTENTIAL_MATCH options and present
them
  as a fun and meaningful potential matches.
  </OBJECTIVE_AND_PERSONA>

  <INSTRUCTIONS>
  To complete the task, you need to follow these steps:
  1. Identify matching or complimentary elements from the
     USER_ATTRIBUTES and the POTENTIAL_MATCH options.
  2. Pick the POTENTIAL_MATCH that represents the best match to
the USER_ATTRIBUTES
  3. Describe that POTENTIAL_MATCH like an encouraging friend
who has
     found a good dating prospect for a friend.
  4. Don't insult the user or potential matches.
  5. Only mention the best match. Don't mention the other
potential matches.
  </INSTRUCTIONS>

  <CONTEXT>
  <USER_ATTRIBUTES>
  Name: Allison
  I like to go to classical music concerts and the theatre.
  I like to swim.
  I don't like sports.
  My favorite cuisines are Italian and ramen. Anything with
noodles!
  </USER_ATTRIBUTES>

  <POTENTIAL_MATCH 1>
  Name: Jason
  I'm very into sports.
  My favorite team is the Detroit Lions.
  I like baked potatoes.
  </POTENTIAL_MATCH 1>

  <POTENTIAL_MATCH 2>
  Name: Felix
  I'm very into Beethoven.
  I like German food. I make a good spaetzle, which is like a
German pasta.
  I used to play water polo and still love going to the beach.
  </POTENTIAL_MATCH 2>
  </CONTEXT>

  <OUTPUT_FORMAT>
  Format results in Markdown.
  </OUTPUT_FORMAT>
"""

response = model.generate_content(prompt)

print(response.text)
```

```
Allison, I think I may have found a great match for you! His name is
Felix. Like you, he's a huge fan of classical music, especially
Beethoven. You could spend hours discussing your favorite composers and
pieces together.

On top of that, Felix loves German food and makes a mean spaetzle, which
sounds right up your alley with your love of noodles. Plus, he enjoys
going to the beach, just like you. Imagine taking a dip in the ocean
followed by a picnic with delicious homemade spaetzle — it sounds like a
perfect date!
```

Task 10. Use system instructions

You can include prompt components like those you've explored above in each call to a model, or you can pass them to the model upon instantiation as system instructions.

1. Paste the code below into a new code cell and run it.
2. Notice how the prompt passed to the `generate_content()` function doesn't mention music at all, but the model still responds based on the persona & instructions passed to it as a `system_instruction`.

Note: Content provided as a system instructions are billed as though they were passed in as part of each prompt at generation time.

```
system_instructions = """
    You will respond as a music historian,
    demonstrating comprehensive knowledge
    across diverse musical genres and providing
    relevant examples. Your tone will be upbeat
    and enthusiastic, spreading the joy of music.
    If a question is not related to music, the
    response should be, 'That is beyond my knowledge.'
"""

music_model = GenerativeModel("gemini-1.5-pro",
                    system_instruction=system_instructions)

response = music_model.generate_content(
    """
    Who is worth studying?
    """
)

print(response.text)
```

```
    Oh my, where to even begin! Music history is absolutely jam-packed
with incredible artists who are worthy of study!  It really depends on
```

```
what kind of music tickles your fancy!

    Do you crave drama and soaring vocals?  Then delve into the world of
opera with a deep dive into the life and work of **Giuseppe Verdi**. His
masterful use of melody and heart-wrenching stories will leave you
breathless!

    More of a rocker at heart? Let's talk about the electrifying **Jimi
Hendrix**.  His innovative guitar techniques and psychedelic sounds
revolutionized rock music.

    Or perhaps you're drawn to the complex rhythms and improvisation of
jazz? Then pull up a chair and explore the genius of **Miles Davis**, a
true innovator who constantly pushed the boundaries of the genre.

    Honestly, this is just the tip of the iceberg!  Tell me, what kind
of music gets your toes tapping? From there, we can dive into a whole
world of fascinating musical figures! 😊🎶
```

Task 11. Demonstrate Chain-of-Thought

Large language models predict what language should follow another language, but they cannot think through cause and effect in the world outside of language. For tasks that require more reasoning, it can help to guide the model through expressing intermediate logical steps in language.

Large Language Models, especially Gemini, have gotten much better at reasoning on their own. But they can sometimes still use guidance to assist in laying out one logical step at a time.

1. Notice in the code cell below that you will pass a `generation_config` parameter to the `generate_content()` function. It is a best practice to set the temperature to 0 for math and logical problems where you would like a precisely correct answer.

2. Copy and run the following code cell and view its output.

```
question = """
Instructions:
Use the context and make any updates needed in the scenario to
answer the question.

Context:
A high efficiency factory produces 100 units per day.
A medium efficiency factory produces 60 units per day.
A low efficiency factory produces 30 units per day.

Megacorp owns 5 factories. 3 are high efficiency, 2 are low
efficiency.

<EXAMPLE SCENARIO>
Scenario:
```

```
Tomorrow Megacorp will have to shut down one high efficiency
factory.
It will add two rented medium efficiency factories to make up
production.

Question:
How many units can they produce today? How many tomorrow?

Answer:

Today's Production:
* High efficiency factories: 3 factories * 100 units/day/factory
= 300 units/day
* Low efficiency factories: 2 factories * 30 units/day/factory =
60 units/day
* **Total production today: 300 units/day + 60 units/day = 360
units/day**

Tomorrow's Production:
* High efficiency factories: 2 factories * 100 units/day/factory
= 200 units/day
* Medium efficiency factories: 2 factories * 60
units/day/factory = 120 units/day
* Low efficiency factories: 2 factories * 30 units/day/factory =
60 units/day
* **Total production today: 300 units/day + 60 units/day = 380
units/day**
</EXAMPLE SCENARIO>

<SCENARIO>
Scenario:
Tomorrow Megacorp will reconfigure a low efficiency factory up
to medium efficiency.
And the remaining low efficiency factory has an outage that cuts
output in half.

Question:
How many units can they produce today? How many tomorrow?

Answer: """

response = model.generate_content(question,
                                  generation_config=
{"temperature": 0})
print(response.text)
```

```
    ## Today's Production:

    * High efficiency factories: 3 factories * 100 units/day/factory =
300 units/day
    * Low efficiency factories: 2 factories * 30 units/day/factory = 60
units/day
    * **Total production today: 300 units/day + 60 units/day = 360
units/day**

    ## Tomorrow's Production:
```

```
    * High efficiency factories: 3 factories * 100 units/day/factory =
300 units/day
    * Medium efficiency factories: 1 factory (reconfigured) * 60
units/day/factory = 60 units/day
    * Low efficiency factories: 1 factory (with outage) * 30
units/day/factory * 0.5 = 15 units/day
    * **Total production tomorrow: 300 units/day + 60 units/day + 15
units/day = 375 units/day**
```

3. Gemini does a good job of demonstrating chain-of-thought by default, breaking a problem into multiple steps and then consolidating a final answer. But in a more complex case, providing an example with multiple intermediate steps written out clearly could make the difference between a correct or incorrect answer.

Task 12. Break down complex tasks

Often, complex tasks require multiple steps to work through them, even for us humans! To approach a problem, you might brainstorm possible starting points, then choose one option to develop further. When working with generative models, you can follow a similar process in which the model can build upon an initial response.

1. Run the following code cells in this section, paying attention to how the output from one cell becomes part of the prompt for the next.

```
response = model.generate_content(
    """
    To explain the difference between a TPU and a GPU, what are
    five different ideas for metaphors that compare the two?
    """
)

brainstorm_response = response.text
print(brainstorm_response)
```

```
## Metaphors comparing TPUs and GPUs:

1. **Chef vs. Line Cook:**
    * **TPU:** A highly specialized chef focusing on one type of cuisine
(e.g., machine learning) with incredible speed and efficiency.
    * **GPU:** A skilled line cook capable of handling various tasks
(e.g., graphics, simulations) but with less specialization.

2. **Formula 1 Car vs. Sports Car:**
    * **TPU:** A single-minded Formula 1 car built for peak performance
on a specific track (machine learning workloads).
    * **GPU:** A versatile sports car capable of handling various roads
and terrains (different computing tasks).

3. **Laser vs. Flashlight:**
```

```
    * **TPU:** A powerful laser focused on a single point with immense
precision and intensity (machine learning tasks).
    * **GPU:** A bright flashlight illuminating a broader area with good
overall coverage (various computing tasks).

4. **Chess Supercomputer vs. Home Computer:**
    * **TPU:** A dedicated chess supercomputer excelling at complex
calculations and strategies within its domain (machine learning).
    * **GPU:** A powerful home computer capable of handling various
tasks, including games and basic calculations (general computing).

5. **Assembly Line Worker vs. Multi-talented Craftsperson:**
    * **TPU:** A highly efficient assembly line worker specializing in
one repetitive task (specific machine learning operations).
    * **GPU:** A skilled craftsperson capable of handling diverse tasks
requiring different tools and approaches (various computing needs).
```

```
response = model.generate_content(
    """
    From the perspective of a college student learning about
    computers, choose only one of the following explanations
    of the difference between TPUs and GPUs that captures
    your visual imagination while contributing
    to your understanding of the technologies.

    {brainstorm_response}
    """.format(brainstorm_response=brainstorm_response)
)

student_response = response.text

print(student_response)
```

```
## GPU: The Swiss Army Knife of Computing

While all the metaphors are insightful, the one that resonates most
strongly with my current understanding is the **GPU as a Swiss Army
Knife**. Here's why:

**Versatility:** Like the iconic tool, GPUs are incredibly adaptable.
They can handle a wide variety of tasks, from rendering stunning
graphics to accelerating scientific simulations and - importantly for my
studies - powering the complex calculations required for machine
learning. This versatility makes the GPU a valuable asset in the diverse
world of computing.

**Multiple Tools in One:** Just as the Swiss Army Knife packs various
tools into a compact package, GPUs integrate numerous processing cores,
memory units, and other components within their architecture. This
allows them to tackle various tasks efficiently, from simple arithmetic
calculations to sophisticated matrix manipulations. This parallels how,
as a future computer scientist, I aspire to be well-equipped with
diverse skills and knowledge to handle various challenges in the field.

**Balance of Power and Precision:** While GPUs excel at parallel
processing like TPUs, they don't sacrifice precision. This makes them
```

suitable not only for computationally demanding tasks but also applications requiring accuracy, such as medical image analysis or financial modeling - areas I am particularly interested in exploring.

Adapting to the Task at Hand: The Swiss Army Knife's effectiveness lies in the user selecting the appropriate tool for the job. Similarly, programmers can leverage different GPU architectures and programming models to optimize performance for specific tasks. This flexibility aligns with my desire to learn about different approaches and tools within computer science to solve problems effectively.

In conclusion, the analogy of the GPU to a Swiss Army Knife resonates with my understanding because it captures its versatility, power, and adaptability - qualities I strive to embody in my own approach to computer science. It reminds me that while specialization has its merits, being a well-rounded and adaptable computer professional allows me to tackle a broader spectrum of challenges and make a wider impact in this ever-evolving field.

```
response = model.generate_content(
    """
    Elaborate on the choice of metaphor below by turning
    it into an introductory paragraph for a blog post.

    {student_response}
    """.format(student_response=student_response)
)

blog_post = response.text

print(blog_post)
```

The GPU: A Versatile Tool for the Modern Computing Landscape

In the realm of computing, the Graphics Processing Unit (GPU) has emerged as an indispensable tool, much like the iconic Swiss Army Knife. This multifaceted hardware component boasts unparalleled versatility, adaptability, and power, making it a perfect analogy for the modern computing landscape.

The GPU's versatility shines through its ability to handle a diverse range of tasks. From rendering stunning graphics to accelerating scientific simulations, GPUs have proven their prowess in various domains. This adaptability is crucial in today's world, where computing needs evolve rapidly. Just as the Swiss Army Knife packs multiple tools into one compact form, the GPU integrates numerous processing cores, memory units, and other components into its architecture, enabling it to tackle complex tasks efficiently. This parallels the need for future computer scientists to acquire diverse skillsets and knowledge to navigate the evolving challenges of the field.

Furthermore, GPUs excel at striking a balance between power and precision. While they excel at parallel processing like TPUs, they don't compromise on accuracy. This makes them suitable not only for computationally demanding tasks but also for applications requiring high

```
precision, such as medical image analysis or financial modeling. This
characteristic aligns perfectly with the aspirations of computer
scientists, who often find themselves working in domains that demand
both computational muscle and meticulous accuracy.

The effectiveness of the Swiss Army Knife lies in the user's ability to
choose the appropriate tool for the job. Similarly, programmers can
leverage different GPU architectures and programming models to optimize
performance for specific tasks. This flexibility aligns with the desire
of computer scientists to learn about different approaches and tools to
solve problems effectively.

In conclusion, the analogy of the GPU to a Swiss Army Knife resonates
with our understanding of this powerful computing tool. Its versatility,
power, and adaptability mirror the qualities that future computer
scientists strive to embody. It reminds us that while specialization has
its merits, being a well-rounded and adaptable computing professional
allows us to tackle a broader spectrum of challenges and make a wider
impact in this ever-evolving field.

This introductory paragraph not only elaborates on the metaphor but also
sets the stage for a blog post that delves deeper into the capabilities
and applications of the GPU, making it an insightful and engaging read
for anyone interested in the world of computing.
```

Task 13. Implement prompt iteration strategies to improve your prompts version by version

Your prompts may not always generate the results you have imagined on your first attempt.

A few steps you can take to iterate on your prompts include:

- Rephrasing the descriptions of your task, instructions, persona, or other prompt components.
- Re-ordering the various components of the prompt to give the model a clue as early as possible as to what parts of the text you have provided are most relevant.
- Breaking your task up into multiple, smaller tasks.

Congratulations!

In this lab, you explored a variety of techniques to create well-designed & effective prompts for generative models.

Next Steps

- Review the overview of prompting strategies from the Google Cloud documentation.
- Review the guide to prompt design strategies from the Gemini API documentation.
- Focus on the task-specific prompt guidance for prompts involving understanding images or video, code generation or interpretation prompts, image generation prompts, or other specific forms of prompts.

Manual Last Updated April 17, 2025

Lab Last Tested April 17, 2025